Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
12	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator. 2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names. 3. (For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details. **Part B — Project Work (Hour 2): MiniBank** Until now accounts have lived in a plain array, which is hard to search and grow. In this lab you upgrade MiniBank's storage to the Collections Framework — a HashMap keyed by account number, sorting with a Comparator, and a thread-safe map for concurrent access. This is the Milestone 2 checkpoint. 1. Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll(). 2. Keep a transaction history in an ArrayList<String>. 3. Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator. 4. Make Account implement Comparable<Account> by comparing account numbers in compareTo(). 5. In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe. 6. In main, add several accounts to the store and print them sorted by balance. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is the difference between a List, a Set and a Map? 2. When do you use a HashMap versus a TreeMap? 3. What is the difference between Comparable and Comparator? **Supplementary Problems –** • Add a method that returns all accounts with a balance above a given value. • Use a TreeMap so the accounts are kept in sorted key order. **Key Skills to be addressed –** List, Set and Map; HashMap and ArrayList; iterating; sorting with Comparable and Comparator; thread-safe collections. **Applications –** Almost every program stores and looks up data in collections; choosing the right one is a core engineering skill. **Learning Outcome –** The student can store, retrieve and sort objects using the Collections Framework. (CO6)	**CO6** PO5, PSO1, PSO2
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Several accounts are added to the store in code.	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	

Practical Number		CO/PO
	Post Laboratory Work Description: Commit and tag lab-12. Confirm look-up by account number is correct and the sorted list is in the right order. Prepare for the Milestone 2 demo (persistence + collections).	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Map-based store and ArrayList history – 35%; sorting with Comparator/Comparable – 25%; thread-safe collection – 15%; naming conventions and viva – 25%.	
	Expected Input / Process / Output **Input:** In code: three accounts with balances 5000, 2000 and 9000 added to the store. **Process:** get() looks an account up by its number in the HashMap; sortedByBalance() copies the values and sorts them with a Comparator. **Expected Output:** get("AC0002") returns the correct account; the sorted list prints the accounts in the order 9000, 5000, 2000.	